SC3103
Embedded Programming

Lecture 3: Inter-Process Communication

Loke Yuan Ren
yrloke@ntu.edu.sg
S3-B2c-114

Inter-Process Communication (IPC)

- A mechanism that allows processes to exchange data and coordinate execution.
- Processes run in separate memory spaces
  - need IPC to share information or synchronize their actions
  1. Pipes
     - Anonymous Pipe
     - Named Pipe
  2. Sockets
  3. Message Queue
  4. Memory Mapping

Anonymous Pipes

The **pipe** in shell can be used to pass information between two running processes

- where the stdout of one process is redirected to the stdin of another process using the pipe command '|'

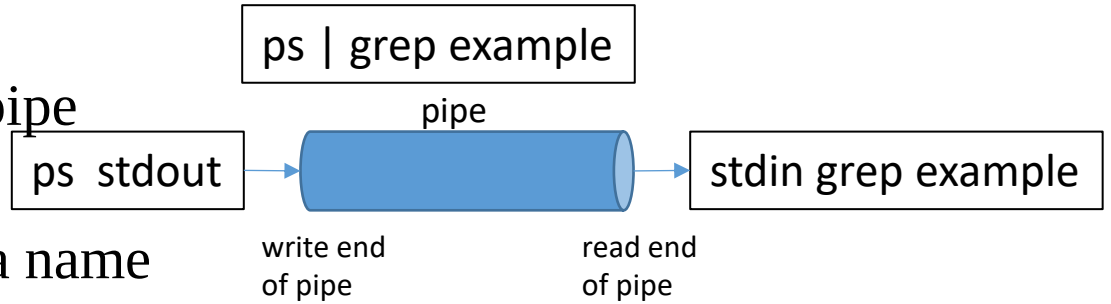
p1 | p2

Both processes are not aware of the existence of the pipe

- hence there is no need to change their code

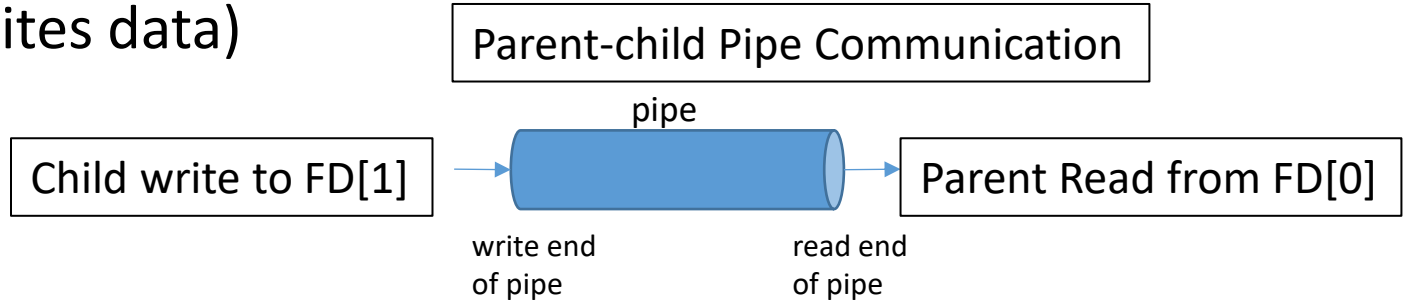
This is called anonymous pipe, which does not have a name

- data flow is unidirectional
- persists only for as long as the processes are running



Anonymous Pipes

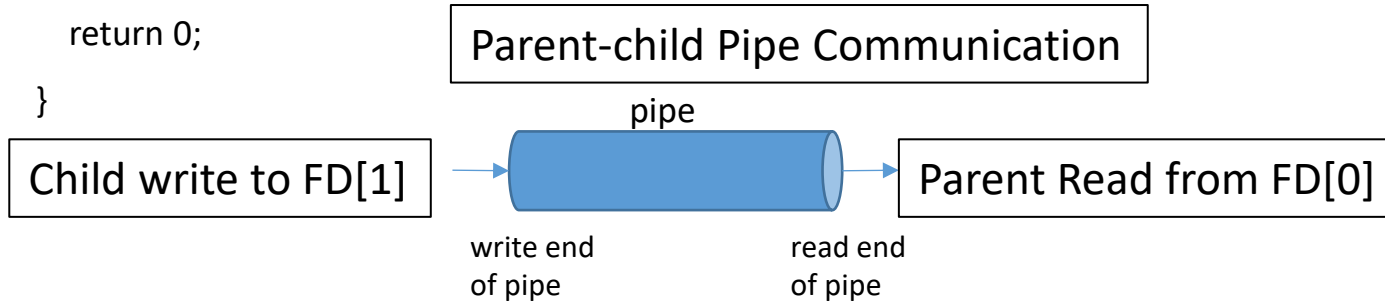
- Key Features:
  - Unidirectional: Data flows in only one direction (from writer to reader).
  - Parent-Child Communication: Used mainly for related processes.
  - Limited to the Same Process Tree: Cannot be used between unrelated processes.
  - Buffer-Based: Data is stored in a **kernel buffer** until read.
- A pipe is represented by a pair of file descriptors (FDs):
  - FD[0] → Read End (process reads data)
  - FD[1] → Write End (process writes data)



Anonymous Pipe in A Multi-process Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
int main() {
    int pfd[2];
    pid_t child_pid;
    // Create the pipe
    if (pipe(pfd) == -1)
        exit(0);

    // Create a child process
    child_pid = fork();
    if (child_pid == -1)
        exit(0);
    if (child_pid == 0) {
        close(pfd[0]);          // Close the read end of the pipe
        char message[] = "Hello, World!\n";
        write(pfd[1], message, strlen(message));
        close(pfd[1]);      // Close the write end of the pipe
        exit(1);
    } else {
        close(pfd[1]); // Close the write end of the pipe
        char buf[256];
        int numRead= read(pfd[0], buf, sizeof(buf));
        printf("Parent process received: %s", buf);
        close(pfd[0]); // Close the read end of the pipe
    }
    return 0;
}
```



IPC
using
Named Pipe

IPC using Named Pipe

- A more **useful and flexible** pipe in Linux is the **named pipe**,
  - also known as **FIFO (First In, First Out)**.
- In Linux, **everything is a file**—including named pipes!
  - **Looks like a file** but doesn't store data permanently.
- **Why Use Named Pipes?**
  - Unlike **anonymous pipes**, **named pipes** can be **shared by multiple processes**.
  - Processes **don't need to run simultaneously**:
  - You can **write** to the pipe **before a reader starts**.
  - **Data may be lost** if no reader is available.
  - **Ideal for server-client communication**.

Named Pipe

A named pipe can be created by using the command mknod or mkfifo from within a program, or at the shell as follows:

mknod /dev/mypipe p

or mkfifo /dev/mypipe

- the named pipe will then appear like a file (i.e. mypipe in the directory /dev in this case) but tagged with “prw…”

- Acts as a communication channel between processes.
- Does not store data; it simply transfers it.
- Blocking by default, but can be made non-blocking (buffered).

Named Pipe – Fifo_read.c and Fifo_write.c

Fifo_read.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_PATH "/tmp/my_fifo"
int main() {
   char buffer[100];
   // Open the FIFO for reading
   int fd = open(FIFO_PATH, O_RDONLY);
   if (fd == -1) {
      perror("open");
      exit(1);    }
   printf("Reader: Waiting for data...\n");
   // Read data from FIFO
   read(fd, buffer, sizeof(buffer));
   printf("Reader: Received: %s\n", buffer);
   // Close FIFO
   close(fd);
   return 0;
}
```

Fifo_write.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_PATH "/tmp/my_fifo"
int main(int argc, char *argv[]) {
   // Create a named pipe (FIFO)
   if (mkfifo(FIFO_PATH, 0666) == -1)
      perror("mkfifo");
   printf("Writer: Waiting for a reader...\n");
   // Open the FIFO for writing
   int fd = open(FIFO_PATH, O_WRONLY);
   if (fd == -1) {
      perror("open");
      exit(1);
   }
   printf("Writer: Writing data...\n");
   // Write data to FIFO
    write(fd, argv[1], sizeof(argv[1]));
   printf("Writer: Data written.\n");
   // Close FIFO
   close(fd);
   return 0;
}
```

Named Pipes: Behavior and Limitations

- Blocking Behavior:
  - Named pipes typically function as streams with blocking behavior.
  - A program waits indefinitely until:
    - Data is available (for reading).
    - Data is read (for writing).
- Non-Blocking Alternative:
  - open(FIFO_PATH, O_RDONLY | O_NONBLOCK);
- **Local-Only Communication:**
  - Named pipes require both communicating programs to access the same filesystem.
  - This means they only work on the same machine (Local IPC).
- Cross-Machine Communication:
  - What if the programs are on different machines or different operating systems?
  - A common alternative is to use sockets for network-based communication.

IPC
using
Socket

Socket

A socket is an application programming interface (API)

- that enable a user program to pass data over the network to another program on another machine

It is probably the most common communication method for processes running on separate machines connected over network

- i.e., remote IPC
- normally using the client-server model.

Socket can also be used between two processes on the same machine, i.e., for local IPC.

- but it is not as efficient as named pipes because it is packet based and there's lots of overheads
- although it is useful in the real world

IPC between Machines

You probably know that computers on the network uses different IP addresses, such as 192.168.1.5

You also know that there can be many different types of internet traffic handled by one machine simultaneously

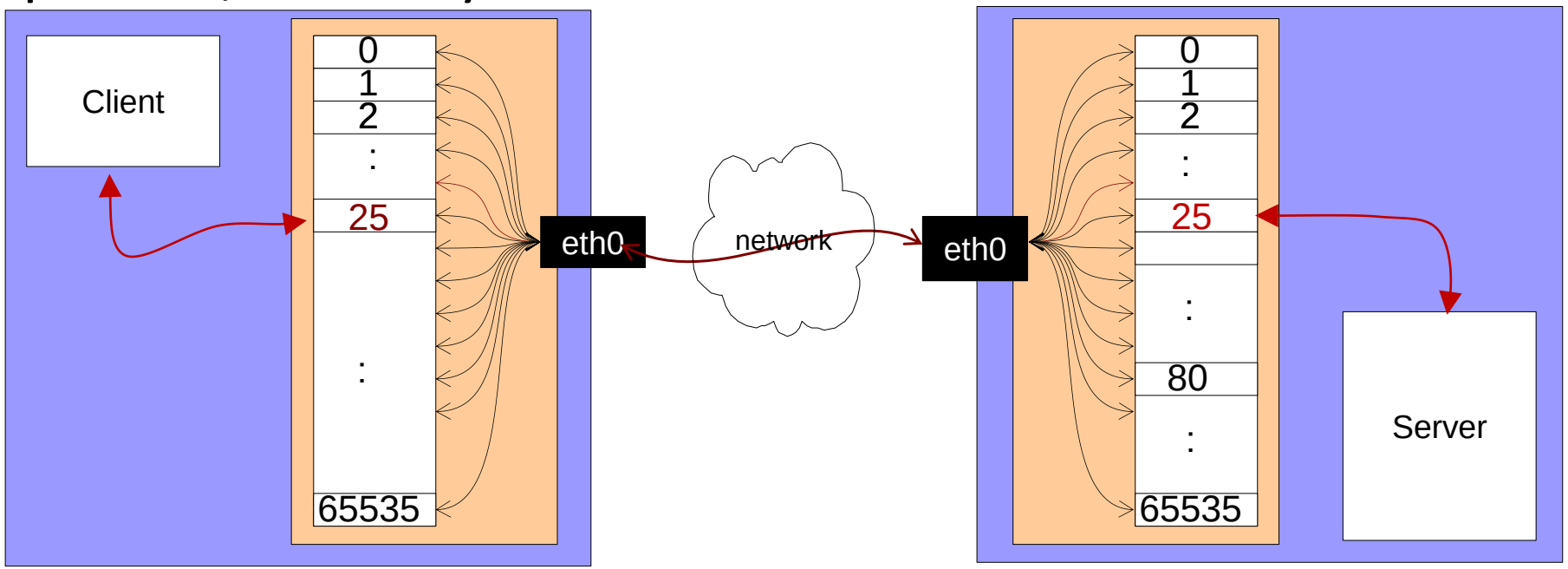
- this works because IP packets contain information to specify their protocol types and ports.

Many Server-Client applications use sockets to establish connections, with commonly used protocol types and their corresponding ports such as

- HTTP - port 80.
- SMTP - port 25.
- Secure Shell - port 22.
- FTP – ports 20(data) and 21(command)

Server-Client IPC

A server is a process that performs some functions on request from a client process, normally over a network



Client computer
157.24.221.56 – 35000
(The 35000 is the source port assigned by the OS)

Server computer
155.69.5.227 - 25
SMTP.ntu.edu.sg
(NTU mail server)

Socket Types

A socket address is identified by the IP address and a port number, and depending on the socket type

data can be transferred using standard protocols such as TCP and UDP

The two common types of socket are

- Stream Socket, which uses TCP provided a reliable, bidirectional, byte-stream channel
- Datagram Socket, which uses UDP in a connectionless manner

To a programmer, a socket looks and behaves much like a low-level file descriptor

- as it is accessed using commands such as read() and write()

We will look at how to send TCP messages from a client to a server.

IP Socket Programming

We're going to develop two (user space) C programs that can communicate by IP sockets

- these can be used to exchange information between different programs on different computers.

We will have a server program running on a computer

- with its IP address 192.168.0.231
- and will be 'listening' on port 12345

We will run a client program on the same computer

- hence having the same IP (or running on ‘localhost’)
- but will write something addressed to port 12345.

Note that this setup is a local IPC

- but can be changed to remote IPC by running the two programs on two machine connected over the network

Aside: Private IP & Privileged Port

IP addresses from 192.168.0.0 to 192.168.255.255 are reserved to be used for devices in closed networks

- would not be available on the public internet
- this range is called Class C private IP addresses, and can have up to 65536 private nodes.

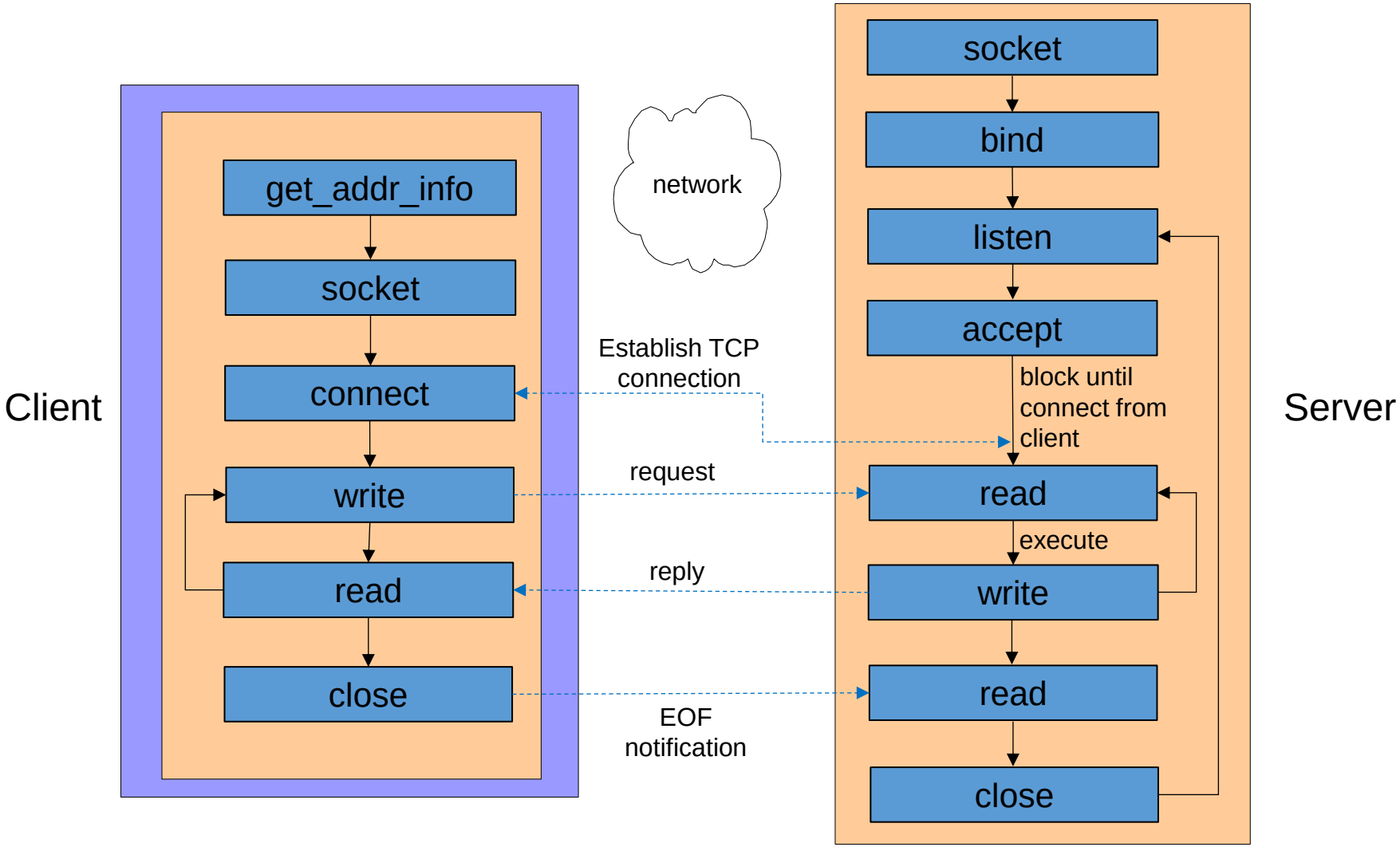
Similarly, TCP/IP port numbers below 1024 are special in that normal users are not allowed to run servers on them

- only root or user with root privilege can start servers binding to these port numbers

Ports with number within the range 0 to 1023 are known as Privileged ports

- as it is ‘safe’ to assume that root can be trusted, so you can log in and send private data to a privileged port safely.

Sockets Interface IPC



Deprecated and not thread-safe

Functions for Socket Programming

```
struct hostent {
        char  *h_name;          /* official name of host */
        char **h_aliases;       /* alias list */
        int    h_addrtype;      /* host address type */
        int    h_length;        /* size of address */
        char **h_addr_list;     /* list of addresses */
    }
```

First, functions used by client program

a) struct hostent * server = gethostbyname(char *)

You call this with an Internet name like www.bbc.com

- and, if it can find the name, it returns a pointer to a hostent structure that includes one or more IP addresses.

If you pass it an IP address, e.g. “155.69.142.100”

- it will just return this in the structure.

Functions for Socket Programming

```
struct addrinfo {
        int             ai_flags;
        int             ai_family;
        int             ai_socktype;
        int             ai_protocol;
        socklen_t       ai_addrlen;
        struct sockaddr *ai_addr;
        char            *ai_canonname;
        struct addrinfo *ai_next;
    };
```

First, functions used by client program

a) int getaddrinfo(const char *restrict node,
const char *restrict service,
const struct addrinfo *restrict hints,
struct addrinfo **restrict res);

You call this with an Internet name like www.bbc.com

- and, if it can find the name, it returns a pointer to a struct addrinfo structure that includes one or more IP addresses.

If you pass it an IP address, e.g. “155.69.142.100”

- it will just return the IP address to the structure.

Functions for Socket Programming

b) sockfd = socket(AF_INET, SOCK_STREAM, 0)

This creates an empty Internet (AF_INET) socket, and specifies the type and transport protocol. (e.g. IPv4 for AF_INET)

In the above

- we open a stream socket (SOCK_STREAM) which is a bit like a character driver
- the argument 0 use default protocol, which is TCP for AF_INET and SOCK_STREAM.
- the return value, sockfd, is an integer identifier.

Functions for Socket Programming

c) connect(sockfd, &serv_addr, sizeof(serv_addr))

This tries to connect an available socket on a server.

You pass in a structure called sockaddr_in (which we need to populate with the server address and port number).

The value of sockfd is enough to identify the socket as the recipient or source of the information.

d) read( ) and write( )

Exactly the same functions we use to read from and write to a file (when using file descriptor)

Socket IPC – Client.c

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <string.h>

void error(char *m)
{   perror(m);
    exit(0);
}

int main(int argc, char *argv[ ])
{ int sockfd, port, n;
  struct sockaddr_in serv_addr;
  struct hostent *server;
  char buffer[256];
```

```
if (argc < 3) error("usage client [hostname] [port]\n");
port = atoi(argv[2]);
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0)  error("ERROR opening socket");
server = gethostbyname(argv[1]);
if (server == NULL)  error("ERROR, no such host\n");
// start with a clean address structure
memset((char *) &serv_addr,  0, sizeof(serv_addr));

serv_addr.sin_family = AF_INET; // internet socket

bcopy((char *)server->h_addr,
     (char*)&serv_addr.sin_addr.s_addr,
     server->h_length);
serv_addr.sin_port = htons(port);
```

Socket IPC – Client.c

```
if (argc < 3) error("usage client [hostname] [port]\n");
port = atoi(argv[2]);
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0)  error("ERROR opening socket");
server = gethostbyname(argv[1]);
if (server == NULL)  error("ERROR, no such host\n");
// start with a clean address structure
memset(((char *) &serv_addr,  0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET; // internet socket
bcopy((char *)server->h_addr,
     (char*)&serv_addr.sin_addr.s_addr,
     server->h_length);
serv_addr.sin_port = htons(port);
if (connect(sockfd,&serv_addr,sizeof(serv_addr)) < 0)
   error("ERROR connecting");
printf("Please enter the message: ");
fgets(buffer,255,stdin);
n = write(sockfd,buffer,strlen(buffer));
if (n < 0)
   error("ERROR writing to socket");
n = read(sockfd,buffer,255);
if (n < 0)
    error("ERROR reading from socket");
printf("%s\n",buffer);
close(sockfd);
return 0;
}
```

Socket IPC – client.c (using getaddrinfo())

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <string.h>

void error(char *m)
{   perror(m);
    exit(0);
}
int main(int argc, char *argv[ ])
{
  int sockfd;
  int n;
  const char* port = argv[2];
  const char* hostname = argv[1];
  struct addrinfo hints, *res;
  char buffer[256];

if (argc < 3) error("usage client [hostname] [port]\n");

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0)  error("ERROR opening socket");

memset(&hints,0,sizeof(hints));
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;

int status = getaddrinfo(hostname, port, &hints, &res);
if (status != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n",
gai_strerror(status));
    exit(1);
  }
  if (connect(sockfd,(struct sockaddr *)res-
>ai_addr,sizeof(res->ai_addr)) < 0)
    error("ERROR connecting");
  printf("Please enter the message: ");
n = write(sockfd,"Hello\0",6);
  if (n < 0)
    error("ERROR writing to socket");
  n = read(sockfd,buffer,255);
  if (n < 0)
     error("ERROR reading from socket");
  printf("%s\n",buffer);
  freeaddrinfo(res);
  close(sockfd);
  return 0;
}
```

Functions for Socket Programming

For the server – there are a number of other useful functions will be including:

a) bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr))

This ties a socket (already created) to a particular address and port. In this case it's the address of the current machine (can have one address per Internet interface).

b) listen(sockfd,2);

This instruction causes the program to inform the OS that it is interested in new connections connects to the socket.

(the program returns immediately after listen)

- 2 = backlog, number of incoming connections that can be queued for acceptance

Functions for Socket Programming

c) newsockfd = accept(sockfd, (struct sockaddr *) &cli_addr, &clilen);

The accept() will block until an incoming connection has been detected on the server's socket. This instruction then accepts the incoming message.

The program typical starts a new task (process or thread) that is bind to a new socket to handle the connection, e.g.:

pthread_create(&thread1, NULL, process, (void*) newsockfd);

Each new socket connection is identified by the combination of IP addresses and port numbers used by the server and the client

i.e. a 4-tuple identification

- hence multiple clients can connect to the same port on the server at the same time

We then read( ) from and write( ) to the new socket.

Socket IPC – Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <string.h>
#include <unistd.h>
void error(char *m)
{
  perror(m);
}

int main(int argc, char *argv[])
{ int sockfd, newsockfd, port, clilen, n;
  char buffer[256];
  struct sockaddr_in serv_addr, cli_addr;
```

```
if (argc < 2)
    error("ERROR, no port provided\n");
  port = atoi(argv[1]);
  sockfd = socket(AF_INET, SOCK_STREAM, 0);
  if (sockfd < 0)
    error("ERROR opening socket");

  memset(((char *) &serv_addr, 0, sizeof(serv_addr));

  serv_addr.sin_family = AF_INET;
  serv_addr.sin_addr.s_addr = INADDR_ANY;
  serv_addr.sin_port = htons(port); //host to network

  if (bind(sockfd, (struct sockaddr *)&serv_addr,
      sizeof(serv_addr)) < 0)
      error("ERROR binding to socket");
```

Socket IPC – Server.c

```
listen(sockfd,2);
  clilen = sizeof(cli_addr);

  newsockfd = accept(sockfd,(struct sockaddr *)
                                        &cli_addr, &clilen);
  if (newsockfd < 0) error("ERROR on accept");

  n = read(newsockfd,buffer,255);
  if (n < 0) error("ERROR reading from socket");

  printf("Message received: %s\n",buffer);

  n = write(newsockfd,"Message acknowledged",21);
  if (n < 0)
     error("ERROR writing back to socket");

  return 0;
}
```

Test Run

```
./server  12345
```

```
./client  localhost  12345

or

./client  192.168.0.231 12345
```

Now you can test both the client and server

either on the same machine (i.e. using localhost)

or on two machines connected by network

(e.g. assuming with IP 192.168.0.231)

Choose a port number that is not of the privileged port!!

Note: you must run server first to open the port, otherwise the client will receive a Connection refused message

IPC
using
Message Queue

Message Queue

Message queue is used to pass ‘messages’ between processes or threads

- one process or thread puts a message into the queue
- the message is held in the queue until another process or thread asks for the message.

Message queue’s underlying operation

- when a message is sent, it is copied from the sender's buffer to some reserved memory space
- when the message is received, it is copied into the buffer supplied by the receiving process or thread.
- i.e. no sharing of memory between the processes.

Compare to a Named pipe that just sends data, without a defined structure

- Message queue has an explicit length and defined type.

Usage of Message Queue

Message queues can be used in a variety of scenarios

- to implement the “producer-consumer” model of cooperating processes or threads.

  (Producer sends its output to the queue; consumer receives the data from the queue.

  When one process gets ahead of the other, it is automatically suspended on the queue until the other process catches up)

- to dispatch units of work to waiting processes or threads (i.e. server-client)

(a server process can direct message traffic between clients on its queue by using the client process PID as the message type – see later.)

POSIX Message Queue

- POSIX message queue's functions (start with prefix "mq_"):
  - mq_open() : Create a queue if it does not exist, and gain access to it
  - mq_getattr() : Get information about an opened queue.
  - mq_setattr() : change the attribute of an opened queue(blocking/nonblocking).
  - mq_send() : send a message to a queue
  - mq_receive() : receive a message from the queue (the message will be removed)
  - mq_notify() : request for asynchronous notification of a message on a queue
  - mq_close() : give up access to a queue.
  - mq_unlink(): remove the queue from the system.

POSIX Message Queue

The header file to be included: <mqueue.h>.

Programs using POSIX message queues (on Linux) must link with the real-time library, librt, using the compiler option -lrt.

The queue descriptor structure type declaration: mqd_t

The attribute of the queue structure type declaration: mq_attr

Example: To open (or create) a message queue (for write)

```
mqd_t queue_d;
struct mq_attr attr;
attr.mq_flags = 0;  // initialize the queue attributes
attr.mq_maxmsg = 10; //max number of messages in the queue
attr.mq_msgsize = MAX_SIZE;
attr.mq_curmsgs = 0;
queue_d = mq_open(/event_queue, O_WRONLY|O_CREAT,
            0660, &attr);
```

POSIX Message Queue

If mq_open() call is successful, the message queue descriptor is returned, which is used in subsequent calls for the queue.

E.g. To send a message to the message queue

```
char msg[MSG_SIZE] // buffer for message
int msg_prio = 0;  // message priority = 0
mq_send(queue_d, msg, MSG_SIZ, msg_prio);
```

Flag	Description
O_CREAT	Create queue
O_EXCL	With O_CREAT, create queue exclusively
O_RDONLY	Reading only
O_WRONLY	Writing only
O_RDWR	Read and Write
O_NONBLOCK	Nonblocking mode

Messages are ordered within the queue in descending order of priority (0 = lowest priority)

when a new message is added to the queue, it is placed after any other messages of the same priority.

If the queue is full, mq_send normally will block until there is space on the queue (unless the O_NONBLOCK flag is enabled, in which case mq_send() returns immediately with errno equal to EAGAIN.)

POSIX Message Queue

To receive a message from the message queue:

```
char buffer[MAX_SIZE + 1];
ssize_t bytes_read;
bytes_read = mq_receive(queue_d, buffer, MAX_SIZE, NULL);
/* add a null character to terminate the string received in the
buffer*/
buffer[bytes_read] = '\0';
```

To close the message queue:

```
mq_close(queue_d);
```

To delete the message queue from the system:

```
mq_unlink(queue_d);
```

Example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <mqueue.h>

#define QUEUE_NAME "/hello_queue"
#define MAX_MSG_SIZE 1024

mqd_t mq;
struct mq_attr attr;
pthread_t sender_thread, receiver_thread;
```

```
void* sender_f (void* arg) {
    while (1) {
        char message[MAX_MSG_SIZE];
        printf("Enter a message (or type quit): ");
        fgets(message, MAX_MSG_SIZE, stdin);
        if (strncmp(message, “quit", 4) == 0) {
            break;
        }
        if (mq_send(mq, message, strlen(message) + 1, 0) == -1) {
            perror("mq_send");
            exit(EXIT_FAILURE);
        }
    }
    pthread_cancel(receiver_thread);
    return NULL;
}
```

```
void* receiver_f (void* arg) {
    char buffer[MAX_MSG_SIZE];

    while (1) {
        ssize_t bytes_received = mq_receive(mq, buffer, MAX_MSG_SIZE, NULL);
        if (bytes_received == -1) {
            perror("mq_receive");
            exit(EXIT_FAILURE);
        }

        printf("Received: %s", buffer);
    }

    return NULL;
}
```

```
int main() {
    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = MAX_MSG_SIZE;
    attr.mq_curmsgs = 0;
    mq = mq_open(QUEUE_NAME, O_CREAT | O_RDWR, 0666, &attr);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        exit(EXIT_FAILURE);
    }

    pthread_create(&sender_thread, NULL, sender_f, NULL);
    pthread_create(&receiver_thread, NULL, receiver_f, NULL);
    pthread_join(sender_thread, NULL);
    pthread_join(receiver_thread, NULL);

    mq_close(mq);
    mq_unlink(QUEUE_NAME);
    return 0;
}
```

Producer-Consumer Problem using POSIX Message Queues

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>
#include <unistd.h>
#include <sys/wait.h>
#define QUEUE_NAME "/my_message_queue"
#define MAX_MSG_SIZE 256
#define MSG_BUFFER_SIZE (MAX_MSG_SIZE + 10)

void producer() {
    mqd_t mq;
    struct mq_attr attr;
    // Set queue attributes
    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;  // Maximum number of messages in the queue
    attr.mq_msgsize = MAX_MSG_SIZE;
    attr.mq_curmsgs = 0;
    // Create or open the message queue
    mq = mq_open(QUEUE_NAME, O_CREAT | O_WRONLY, 0644, &attr);
    if (mq == -1) {
        perror("Producer: mq_open");
        exit(1);
    }

    char *messages[] = {"Message 1", "Message 2", "Message 3", "STOP"};
    for (int i = 0; i < 4; i++) {
        if (mq_send(mq, messages[i], strlen(messages[i]) + 1, 0) == -1) {
            perror("Producer: mq_send");
            exit(1);
        }
        printf("Producer: Sent -> %s\n", messages[i]);
        sleep(1); // Simulate delay
    }
    mq_close(mq);
}

void consumer() {
    mqd_t mq;
    char buffer[MSG_BUFFER_SIZE];
    // Open the message queue
    mq = mq_open(QUEUE_NAME, O_RDONLY);
    if (mq == -1) {
        perror("Consumer: mq_open");
        exit(1);
    }
    while (1) {
        ssize_t bytes_read = mq_receive(mq, buffer, MSG_BUFFER_SIZE, NULL);
        if (bytes_read == -1) {
            perror("Consumer: mq_receive");
            exit(1);
        }
        buffer[bytes_read] = '\0'; // Null-terminate message
        printf("Consumer: Received -> %s\n", buffer);
        if (strcmp(buffer, "STOP") == 0) {
            printf("Consumer: STOP signal received. Exiting.\n");
            break;
        }
    }
    mq_close(mq);
    mq_unlink(QUEUE_NAME); // Delete message queue
}

int main() {
    pid_t pid = fork();
    int status;
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) {
        // Child process -> Consumer
        consumer();
    } else {
        // Parent process -> Producer
        producer();
        wait(&status); // Wait for consumer to finish
        printf("Main process: Message queue deleted.\n");
    }
    return 0;
}
```

mq_notify()

Int mq_notify(mqd_t mqdes, const struct sigevent *sevp)

- to register for notification when a message is available
- i.e. for asynchronous notification of arrival of a message on an empty queue

Use in conjunction with a sigevent structure.

Example:

```
sev.sigev_notify = SIGEV_THREAD;
sev.sigev_notify_function = tfunc;
sev.sigev_notify_attributes = NULL;
sev.sigev_value.sival_ptr = &queue_d;
if (mq_notify(queue_d, &sev) == -1)
    handle_error("mq_notify");
```

Memory Mappings

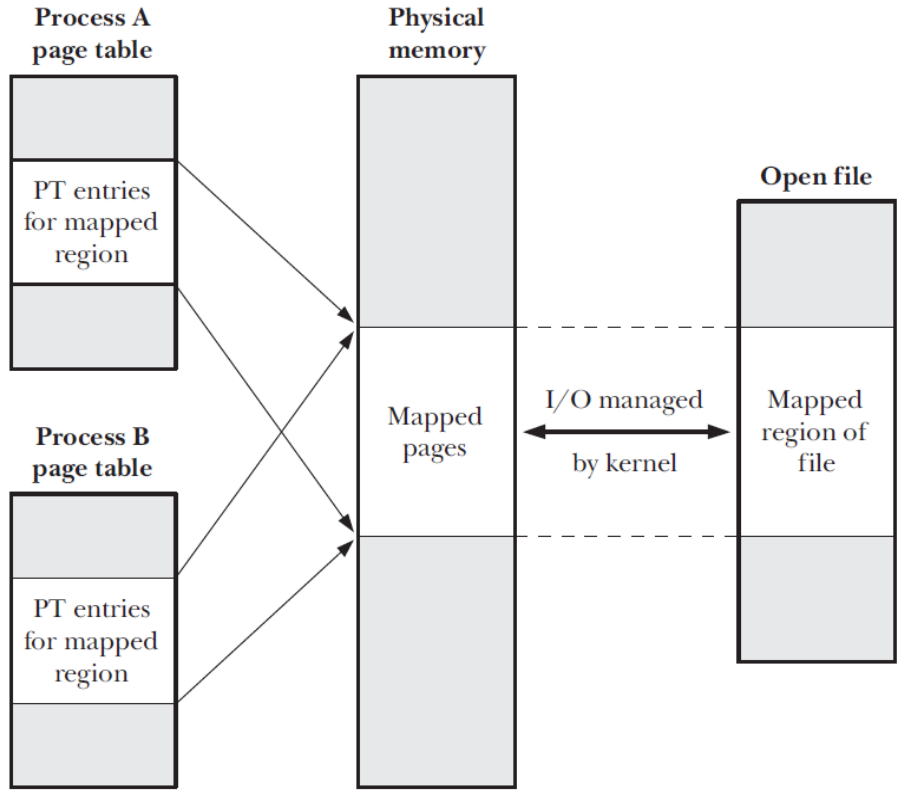
Memory Mapping between Two Processes

- Memory mapping is the process of linking a file or a portion of memory directly into the process's address space. It allows files or devices to be accessed as if they were part of the program's memory.

- Eliminates the need for explicit file I/O operations (read, write), enabling more efficient file handling.

- The memory in one process’s mapping may be shared with mappings in other processes
  - the page-table entries of each process point to the same pages of RAM
  - When two processes map the same region of a file, they share the same pages of physical memory.
  - A child process created by fork() inherits copies of its parent’s mappings, and these mappings refer to the same pages of physical memory as the corresponding mappings in the parent.

Two processes with a shared memory mapping



The mmap() function

- mmap() is a system call that maps files or devices into memory.

void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);

  - addr: Starting address for the mapping.
  - length: Number of bytes to map.
  - prot: Memory protection (e.g., PROT_READ, PROT_WRITE).
  - flags: Flags to control the behavior (e.g., MAP_SHARED, MAP_PRIVATE).
  - fd: File descriptor of the file to map.
  - offset: Starting point in the file to map from.

Example of merge sort by fork()

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <unistd.h>

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int i = 0; i < n2; i++)
        R[i] = arr[mid + 1 + i];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}
```

Example of merge sort by fork()

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        pid_t pid = fork(); // Create a new process using fork

        if (pid == 0) {
            // Child process handles the left half
            mergeSort(arr, left, mid);
            exit(0); // Child process exits after sorting
        } else if (pid > 0) {
            // Parent process handles the right half
            mergeSort(arr, mid + 1, right);
            // Wait for the child process to finish
            wait(NULL);

            // Merge the two sorted halves
            merge(arr, left, mid, right);
        }
    }
}
```

```
int main() {
    int n = 10;
    int *arr = mmap(NULL, n * sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    // Initialize array
    int initArr[] = {38, 27, 99,43, 3, 9, 82, 10,1,78};
    for (int i = 0; i < n; i++) {
        arr[i] = initArr[i];
    }
    printf("Original array: \n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    // Perform parallel merge sort using fork and shared memory
    mergeSort(arr, 0, n - 1);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    munmap(arr, n * sizeof(int));
    return 0;
}
```

Two types of memory mapping

- **File mapping**: A file mapping maps a region of a file directly into the calling process's virtual memory. Once a file is mapped, its contents can be accessed by operations on the bytes in the corresponding memory region. The pages of the mapping are (automatically) loaded from the file as required. This type of mapping is also known as a file-based mapping or memory-mapped file.

```
int fd = open("example.txt", O_RDWR);
char *mapped = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
munmap(mapped, size * sizeof(char));
```

- **Anonymous mapping**: An anonymous mapping doesn't have a corresponding file. (See merge sort example)

When to Use mmap()?

- Sharing memory between processes.
- Working with large files that don’t fit into physical memory.
  - No data copying

Note:

- Not suitable for small files due to overhead.
- Requires careful management to avoid segmentation faults.

Comparison

Feature	Named Pipes (FIFO)	Shared Memory	Message Queues
Communication	Unidirectional (typically)	Bidirectional	Bidirectional
Speed	Slower due to copying of data	Fastest (no data copying)	Slower than shared memory
Synchronization	Implicit FIFO order (no need for locks)	Requires explicit synchronization	Queued messages with possible prioritization
Data Structure	Stream of bytes (like a file)	Direct access to memory	Discrete messages
Persistence	Persist in filesystem until deleted	Does not persist after processes exit	Can be configured to persist
Best Use Case	Simple, unidirectional communication between unrelated processes	Fast, large data transfer, high performance	Complex communication with multiple messages

Summary

We have explored three methods of IPC:

1. Through named pipes
   Useful where both processes are running on the same filesystem.
   This method is very simple
   - it's just like reading from or writing to a file.
2. Using sockets
   Useful for networked machines.
   This is a little more complicated
   - but it is much more practical for many real world client-server based applications.
3. Using message queue
   Use for local IPC through shared memory
4. Memory Mappings